

Chapter 10

Kubernetes in Action 1st Edition

StatefulSets

deploying replicated stateful applications



신재근 (Jagger)

10장의 핵심 질문

Cassandra · MongoDB · Kafka 같은 분산 데이터 시스템을
Kubernetes에서 어떻게 안전하게 운영할까?

답

- StatefulSet — 안정적 이름·DNS·스토리지
- Headless Service — peer discovery용 DNS
- volumeClaimTemplate — Pod별 전용 PVC 자동 생성

왜 ReplicaSet으로는 안 되나 — 3가지 시도

분산 DB를 ReplicaSet으로 운영하려면 어떻게든 Pod별 전용 PVC가 필요합니다. 직관적 접근 3가지를 살펴봅니다.

시도	방법	실패 이유
1	Pod 수동 생성	컨트롤러 관리 X → Pod 죽으면 사람이 매번 복구
2	Pod별 ReplicaSet (1개씩)	Scale = RS 생성/삭제 / Update = RS마다 반복 → 운영 폭증
3	동일 PV + 디렉토리 분할	(a) I/O 병목 (b) 어느 디렉토리를 누가 쓸지 결정 불가

→ 어느 방법도 운영 가능한 수준이 아님 — 새 컨트롤러가 필요

네트워크 Identity 문제 — 데이터만으론 부족

설령 Pod별 전용 PVC를 어떻게든 만들었다 해도, 한 가지 문제가 남습니다.

ReplicaSet이 Pod를 재생성하면 이름·IP가 모두 바뀐다.

# 재기동 전	→ 재기동 후	
hello-dfskwer	hello-abcde123	← 이름 바뀜
10.0.1.5	10.0.3.17	← IP 바뀜

분산 시스템에서 이게 왜 문제인가:

- MongoDB replica set: `rs.add("mongo-1.cluster:27017")` — 이름이 바뀌면 멤버 등록 깨짐
- Kafka: broker.id를 호스트네임에 매핑 — 재기동 후 토픽 라우팅 망가짐
- Zookeeper: `myid` 와 호스트네임 1:1 매핑 — 클러스터가 분열됨

→ 데이터 영속성(PVC) + 네트워크 동일성(이름/DNS) 둘 다 필요 → 이게 StatefulSet의 출발점

책의 비유 — Cattle vs Pet

ReplicaSet/Deployment의 가정:

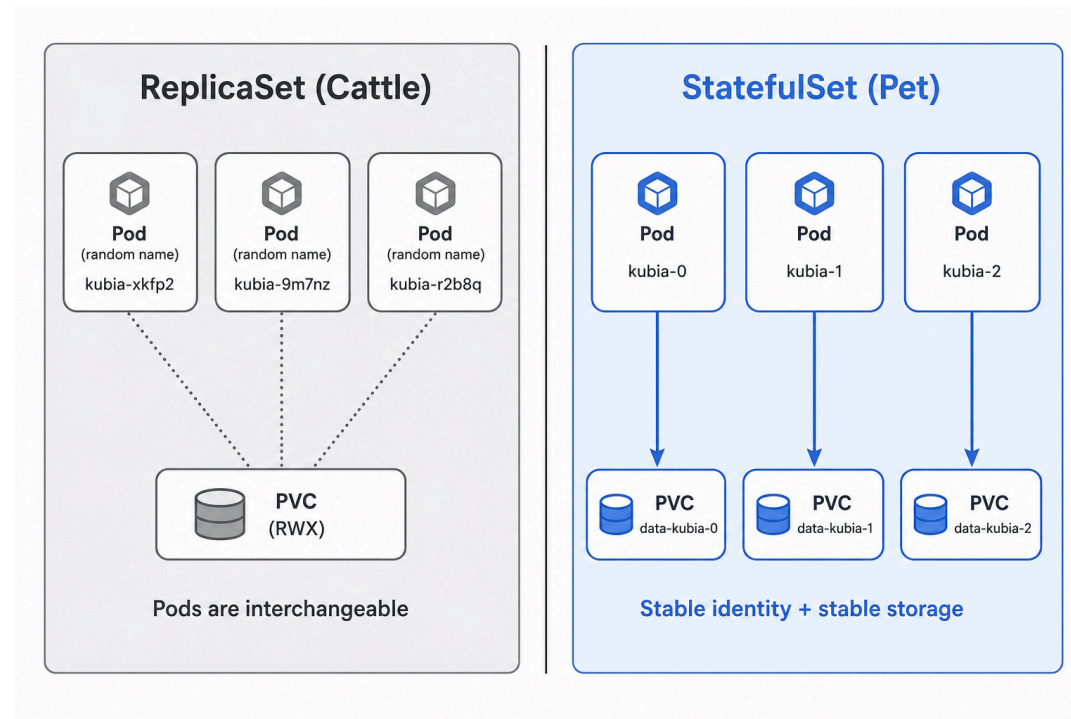
- 모든 Pod이 동일하고 교체 가능 → 소떼(Cattle)
- 한 마리 죽으면 다른 소로 대체 → 무상태 워크로드에 적합

이 가정이 깨지는 워크로드:

- 분산 DB: Cassandra·MongoDB·etcd — 각 노드가 다른 데이터 shard 소유
- 메시지 브로커: Kafka — broker마다 partition 일부 소유
- 마스터/슬레이브: MySQL replication — primary와 replica 역할이 다름

이런 워크로드는 각 Pod이 고유한 정체성과 데이터를 가져야 함 → Pet 모델 필요

Cattle vs Pet — 그림으로



- Cattle: 익명·교체 가능 → 무상태 웹/API
- Pet: 이름·정체성·전용 storage → 분산 DB

StatefulSet 3가지 핵심 보장

보장	내용
Stable Network Identity	Pod 이름이 <code><sts>--<ordinal></code> 로 고정, 개별 DNS 발급
Stable Storage	Pod별 PVC 자동 생성, 재생성 시 재바인딩
Ordered Operations	생성 0→N 순차, 삭제 N→0 역순, 업데이트도 역순

세 보장은 함께 작동. 하나라도 빠지면 corruption/split-brain 위험

구성 요소 1 — Headless Service

책에서 강조하는 부분: **clusterIP: None** 한 줄이 모든 것을 결정

```
apiVersion: v1
kind: Service
metadata:
  name: kuba
spec:
  clusterIP: None          # ★ 일반 ClusterIP 할당 안 함
  selector:
    app: kuba
  ports:
    - port: 80
```

이 한 줄이 가져오는 변화:

- kube-proxy LB 우회 → 단일 ClusterIP 없음
- DNS 쿼리 시 모든 Pod IP 반환 (multiple A records)
- 각 Pod에 개별 DNS 등록 활성화 (`<pod>.<svc>.<ns>...`)
- SRV 레코드 발급 → peer discovery 가능

구성 요소 2 — StatefulSet 본체

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kubia
spec:
  serviceName: kubia          # ★ Headless Service 이름과 일치
  replicas: 2
  selector:
    matchLabels: { app: kubia } # 표준 selector
  template:
    metadata: { labels: { app: kubia } }
    spec:
      containers:
      - name: kubia
        image: luksa/kubia-pet
        ports: [ { containerPort: 8080 } ]
# → volumeClaimTemplates는 다음 슬라이드
```

- `serviceName` 이 DNS 이름의 핵심 ← Headless Service와의 연결고리

구성 요소 3 — volumeClaimTemplate

책의 핵심 아이디어: PVC를 "템플릿"으로 정의 → Pod마다 자동 인스턴스화

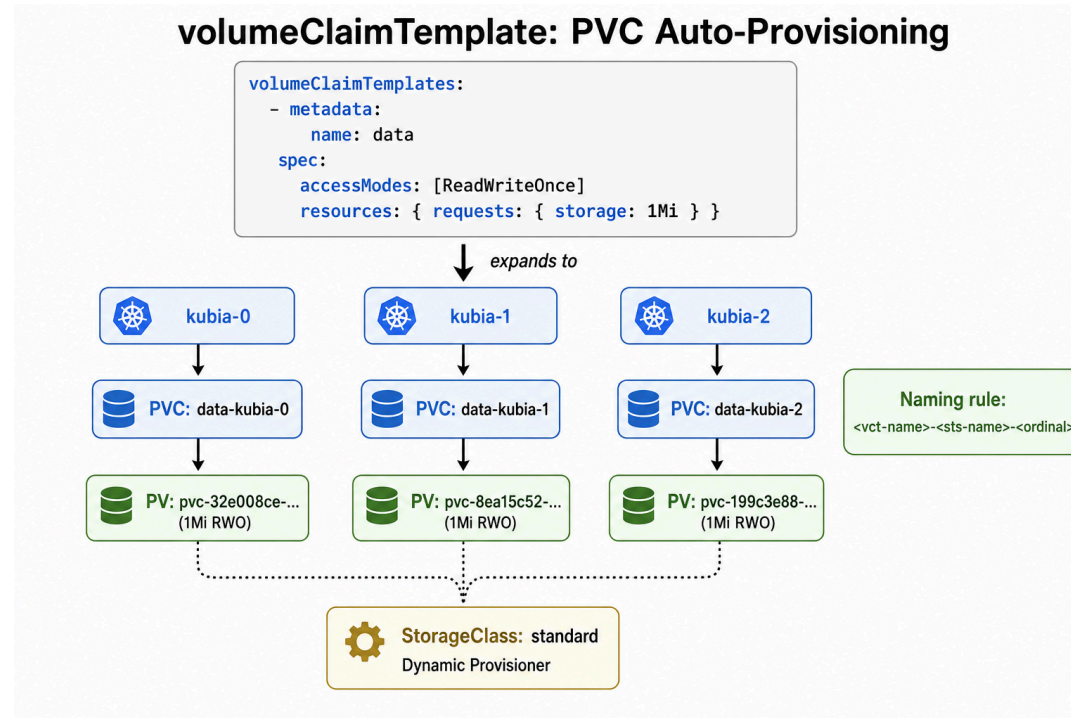
```
spec:
  ...
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources: { requests: { storage: 1Mi } }
```

PVC 명명 규칙: <vct-name>--<sts-name>--<ordinal>

→ data-kubia-0, data-kubia-1, ...

STS를 삭제해도 PVC는 자동으로 사라지지 않음 — 데이터 손실 방지의 핵심 안전장치

volumeClaimTemplate 동작 — 그림



- StorageClass의 동적 프로비저너가 PV를 자동 생성
- Pod ↔ PVC 1:1 매핑 (각 Pod은 자기 ordinal PVC만 마운트)

Demo 1 — 순차 생성 관찰 (1/2): 명령

목표: StatefulSet이 정말 한 Pod씩 순차적으로 시작되는가?

```
$ kubectl apply -f kubia-service-headless.yaml -f kubia-statefulset.yaml
service/kubia created
statefulset.apps/kubia created

# 5초 간격으로 Pod 상태 폴링
$ for i in {1..12}; do
    echo "--- t=$((i*5))s ---"
    kubectl get pods -l app=kubia --no-headers
    sleep 5
done
```

비교 기준: ReplicaSet이었다면 t=0~5초에 두 Pod이 동시에 ContainerCreating

Demo 1 — 순차 생성 (2/3): 설명

관찰 포인트: 다음 슬라이드의 ---- t=20s ---- 블록을 주목

- t=20s 시점: `kubia-0` 만 존재, `kubia-1` 은 아예 생성도 안 됨
- t=25s 시점: `kubia-0` 이 Ready 되자마자 `kubia-1` 처음 등장
- 즉, "Pending"이 아니라 "객체 자체가 미존재" 상태로 대기

무엇을 증명하나

- StatefulSet은 ordinal 순서를 엄격히 직렬화 — Deployment(병렬 생성)와 정반대
- 비용: 총 부팅 시간 ~45s = 2배 (병렬이면 ~25s)
- 보상: MongoDB primary가 secondary보다 먼저 떠나야 하는 등 순서 의존 시스템에 안전
- → 25슬라이드 motivation의 "P2P 시스템엔 과한 비용" 주장의 근거

Demo 1 — 순차 생성 (3/3): 결과

```
--- t=5s ---
kubia-0    0/1    ContainerCreating    0    0s    ← (1) kubia-0만 생성

--- t=20s ---
kubia-0    0/1    ContainerCreating    0    15s   ← (2) 15초 지나도 kubia-1 미존재

--- t=25s ---
kubia-0    1/1    Running              0    20s   ← (3) kubia-0 Ready 전환
kubia-1    0/1    ContainerCreating    0    4s    ← ★ 직후 kubia-1 비로소 시작

--- t=45s ---
kubia-0    1/1    Running              0    41s
kubia-1    1/1    Running              0    25s   ← (4) 둘 다 Ready (총 ~45s)
```

kubia-0 Ready 되기 전엔 kubia-1 이 생성조차 안 됨

Demo 2 — PVC/PV 자동 생성 검증

```
$ kubectl get pvc
NAME                STATUS    VOLUME                                     CAPACITY   AGE
data-kubia-0        Bound     pvc-32e008ce-a543-4b54-997a-72dd459df7ce  1Mi        72s
data-kubia-1        Bound     pvc-8ea15c52-54bd-4e0e-afdd-7ff6637285d1  1Mi        56s

$ kubectl get pv
NAME                                     CAPACITY   RECLAIM POLICY   STATUS   CLAIM                                AGE
pvc-32e008ce-a543-4b54-997a-72dd459df7ce  1Mi        Delete           Bound    default/data-kubia-0               72s
pvc-8ea15c52-54bd-4e0e-afdd-7ff6637285d1  1Mi        Delete           Bound    default/data-kubia-1               56s

$ kubectl get svc kubia
NAME      TYPE        CLUSTER-IP   PORT(S)
kubia     ClusterIP   None         80/TCP
                                                ← ★ Headless
```

이름 규칙 정확히 `data-kubia-<ordinal>`, StorageClass `standard` 가 PV 동적 생성

Stable Storage — 왜 중요한가

책의 핵심 주장:

Pod의 수명과 무관하게 데이터는 보존되어야 한다

분산 DB에서 일어나는 일:

1. Pod이 죽는 일은 흔하다 (노드 재시작, OOM, 강제 종료)
2. 새 Pod이 같은 이름으로 부활해도, 데이터를 찾지 못하면 클러스터 멤버에서 제외됨
3. 다시 복귀하려면 처음부터 데이터 sync 필요 → 비용 큼

StatefulSet의 약속:

- Pod 이름 = ordinal로 고정 (Stable Identity)
- 재생성 시 같은 PVC 자동 재바인딩 (Stable Storage)
- → 새 Pod이지만 옛 데이터 그대로

Demo 3-1: kubia-0에 데이터 쓰기

luksa/kubia-pet 은 POST body를 디스크에 저장 → GET 시 반환

```
$ kubectl run ch10-writer --image=curlimages/curl:8.7.1 --restart=Never --command -- sh -c '
  curl -s -X POST -d "Data stored on kubia-0 at $(date +%s)" \
    http://kubia-0.kubia.default.svc.cluster.local:8080
  curl -s http://kubia-0.kubia.default.svc.cluster.local:8080
  curl -s http://kubia-1.kubia.default.svc.cluster.local:8080'
```

```
writing to kubia-0...
Data stored on pod kubia-0

reading from kubia-0...
You've hit kubia-0
Data stored on this pod: Data stored on kubia-0 at 1779075542

reading from kubia-1...
You've hit kubia-1
Data stored on this pod: No data posted yet
```

← ★ 독립 storage

Demo 3-2 (1/2): 설명 — Stable Identity vs Stable Storage

관찰 포인트: 다음 슬라이드에서 세 가지 값을 비교

비교 항목	before	after	의미
Pod UID	28c99787-...	3fd40eca-...	다른 Pod (객체 재생성)
PVC AGE	2m13s	2m13s	PVC 보존 (디스크 유지)
데이터	1779075542	1779075542	데이터 복귀

무엇을 증명하나 — Stable Identity + Stable Storage 분리 원칙

- Pod = 일회용(UID 바뀜), PVC = 영속(AGE 그대로) → 책의 핵심 분리
- 같은 이름 `kubia-0` 으로 재생성 시 K8s가 PVC `data-kubia-0` 을 자동 재바인딩
- → MongoDB Pod이 죽어도 데이터 디스크는 살아서 새 Pod에 그대로 마운트되는 구조

Demo 3-2 (2/2): 데모 — Pod 삭제 → 재생성, 데이터 검증

```
$ kubectl get pod kubia-0 -o jsonpath='{.metadata.uid}'      # before
28c99787-1753-48fe-99e9-31ec09fb68fc                       ← (1) 옛 Pod UID

$ kubectl delete pod kubia-0 && \
  kubectl wait --for=condition=Ready pod/kubia-0 --timeout=120s
pod/kubia-0 condition met                                   ← (2) 같은 이름으로 재생성

$ kubectl get pod kubia-0 -o jsonpath='{.metadata.uid}'      # after
3fd40eca-ada4-416f-9b76-578a99892b6c                       ← ★ 다른 UID = 새 Pod

$ kubectl get pvc data-kubia-0
data-kubia-0    Bound      pvc-32e008ce-...    1Mi    2m13s    ← ★ AGE 그대로 = PVC 보존

$ curl http://kubia-0.kubia.default.svc:8080
Data stored on this pod: Data stored on kubia-0 at 1779075542 ← ★ 데이터 복귀
```

같은 이름 `kubia-0` 으로 재생성 → K8s가 PVC `data-kubia-0` 을 자동 재바인딩

Headless Service + DNS SRV — 책의 설명

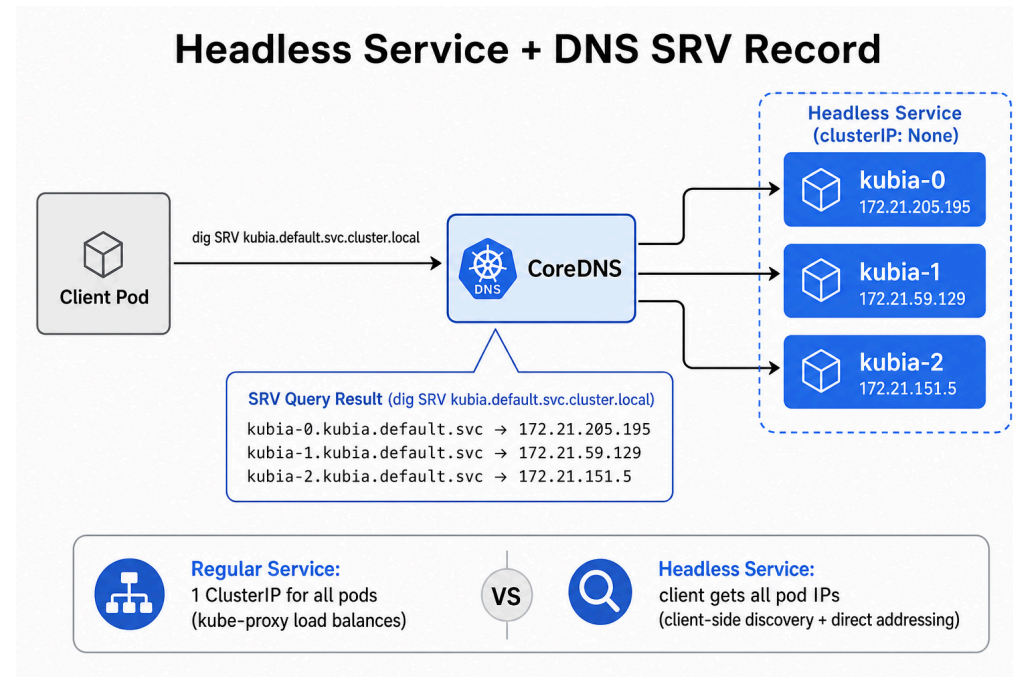
책에서 강조: Headless Service의 DNS는 일반 Service와 근본적으로 다르다

Service 타입	DNS 응답	사용 패턴
Regular (ClusterIP)	단일 ClusterIP	무상태 LB
Headless	모든 Pod IP + 개별 호스트네임 + SRV	Peer discovery, sharding

SRV 레코드가 가져오는 변화:

- 클라이언트가 한 번의 쿼리로 클러스터 전체 멤버 + 포트 정보 획득
- Cassandra seed list 등록, Kafka brokers 발견, etcd member 추가에 사용

Headless Service — 그림



Demo 4-1: DNS SRV 쿼리 명령

```
$ kubectl run ch10-dns --image=nicolaka/netshoot --restart=Never --command -- sh -c '
  echo "--- SRV record ---"
  dig +noall +answer +additional SRV kuba.default.svc.cluster.local

  echo "--- A record: individual pods ---"
  dig +short kuba-0.kuba.default.svc.cluster.local
  dig +short kuba-1.kuba.default.svc.cluster.local

  echo "--- A record: headless service ---"
  dig +short kuba.default.svc.cluster.local

  echo "--- A record: regular service (비교) ---"
  dig +short kuba-public.default.svc.cluster.local'
```

※ tutum/dnsutils 는 containerd v2.x 비호환 → nicolaka/netshoot 사용

Demo 4-2: DNS SRV 쿼리 결과

관찰 포인트: 마지막 두 블록 비교 — Headless는 모든 Pod IP, Regular는 단일 ClusterIP

```
--- SRV record (kubia headless service) ---
kubia.default.svc.cluster.local. 30 IN SRV 0 50 80 kubia-1.kubia.default.svc...
kubia.default.svc.cluster.local. 30 IN SRV 0 50 80 kubia-0.kubia.default.svc...
kubia-0.kubia.default.svc.cluster.local. 30 IN A 172.21.205.195 ← (1) 개별 호스트네임
kubia-1.kubia.default.svc.cluster.local. 30 IN A 172.21.59.129 ← (2) 개별 IP 매핑

--- A record: headless service (all pod IPs) ---
172.21.205.195
172.21.59.129 ← ★ 모든 Pod IP 반환 (LB 없음)

--- Compare: regular service kubia-public ---
172.20.180.141 ← ★ 단일 ClusterIP (kube-proxy LB)
```

무엇을 증명하나 — `clusterIP: None` 한 줄이 만드는 차이

- SRV 레코드: `kubia-0`, `kubia-1` 같은 이름 → IP·Port 자동 등록 → peer discovery 가능
- A 레코드 (headless): 모든 Pod IP 반환 → 클라이언트가 직접 선택해서 연결
- A 레코드 (regular): 단일 ClusterIP → kube-proxy 우회 불가 → 분산 DB의 leader 직접 접속 ✕

Scale 동작 원리 — 책의 설명

StatefulSet의 scale 동작은 방향성을 가진다:

Scale Up (replicas $N \rightarrow N+k$):

- 가장 높은 ordinal부터 순차 추가 (`kubia-N` , `kubia-N+1` , ...)
- 각 Pod이 Ready 된 뒤에야 다음 시작 (OrderedReady)
- 새 PVC 자동 프로비저닝

Scale Down (replicas $N \rightarrow N-k$):

- 가장 높은 ordinal부터 역순 제거 (`kubia-N-1` , ...)
- PVC는 보존 ← 책에서 강조하는 가장 중요한 안전장치

Scale down 후 다시 scale up 하면 같은 ordinal의 PVC 재바인딩 → 데이터 복귀

Demo 5 — Scale Up (kubia-2 추가)

```
$ kubectl scale statefulset kubia --replicas=3
statefulset.apps/kubia scaled
```

```
--- t=3s ---
kubia-0    1/1    Running           0    2m
kubia-1    1/1    Running           0    3m55s
kubia-2    0/1    ContainerCreating 0    0s      ← ★ kubia-2만 신규

--- t=21s ---
kubia-2    1/1    Running           0    19s     ← Ready

$ kubectl get pvc
data-kubia-0    Bound    pvc-32e008ce-...    1Mi    4m42s
data-kubia-1    Bound    pvc-8ea15c52-...    1Mi    4m26s
data-kubia-2    Bound    pvc-199c3e88-...    1Mi    31s     ← ★ 자동 생성
```

기존 Pod·PVC 영향 없음. 새 ordinal과 새 PVC만 추가

Demo 6 — Scale Down + PVC 보존

```
$ kubectl scale statefulset kubia --replicas=1
```

--- t=3s ---

kubia-2	1/1	Terminating	0	41s	← ★ 가장 높은 ordinal부터
kubia-1	1/1	Running	0	4m36s	

--- 잠시 후 ---

kubia-1	1/1	Terminating			← ★ 그 다음 kubia-1
---------	-----	-------------	--	--	------------------

=== 최종 ===

kubia-0	1/1	Running			← kubia-0만 남음
---------	-----	---------	--	--	---------------

=== ★ PVC: 3개 모두 보존 ★ ===

data-kubia-0	Bound	...	32m	
data-kubia-1	Bound	...	32m	← 삭제 안 됨
data-kubia-2	Bound	...	28m	← 삭제 안 됨

At-Most-One Semantic — 불변조건

StatefulSet의 가장 중요한 불변조건:

같은 ordinal의 Pod이 동시에 두 개 존재해선 안 된다

구체 시나리오 — Kafka 브로커가 토픽 복제 중일 때:

- 브로커 kafka-2 제거 → 토픽 C 파티션을 kafka-1 로 복제 시작
- 만약 kafka-1 · kafka-2 가 동시 삭제되면 → 토픽 C 데이터 영구 손실
- 단순 corruption이 아니라 복제 in-flight 단계의 데이터 유실 → 더 위험

→ 이 때문에 StatefulSet은 "확실히 한 번에 한 Pod만" 처리

노드 장애 시 K8s가 취하는 동작:

1. 노드 응답 없음 → NotReady 마크
2. 기본 pod-eviction-timeout: 5분 까지 새 Pod 안 만들
3. timeout 후에야 새 Pod 생성 → 책에서 "느려 보이는" 이유

At-Most-One Semantic — 함정과 위험

위험한 명령:

```
kubectl delete pod kubia-0 --force --grace-period=0
```

언제 split-brain이 발생하나:

- 노드가 사실 살아있고 네트워크만 격리된 상태
- 강제 삭제로 새 `kubia-0` 이 다른 노드에 생성됨
- 격리 해제되면 두 개의 `kubia-0` 이 같은 PVC에 동시 접근
- → 즉시 데이터 corruption

결론:

- `--force --grace-period=0` 은 노드가 확실히 죽었음을 사람이 보증할 때만
- 분산 DB(MongoDB·etcd)는 quorum 보호 (`PDB`)로 함께 막아야 안전

1판 → 2판 변화 ① — podManagementPolicy

1판의 한계

- 1판은 `OrderedReady` 모드만 다룸 (실제로는 기본값이고 유일했음)
- Pod이 N개면 모두 Ready되는 데 N배 시간 소요
- 분산 DB라도 P2P 시스템(Cassandra·etcd)은 순서가 무의미한데 강제로 기다림

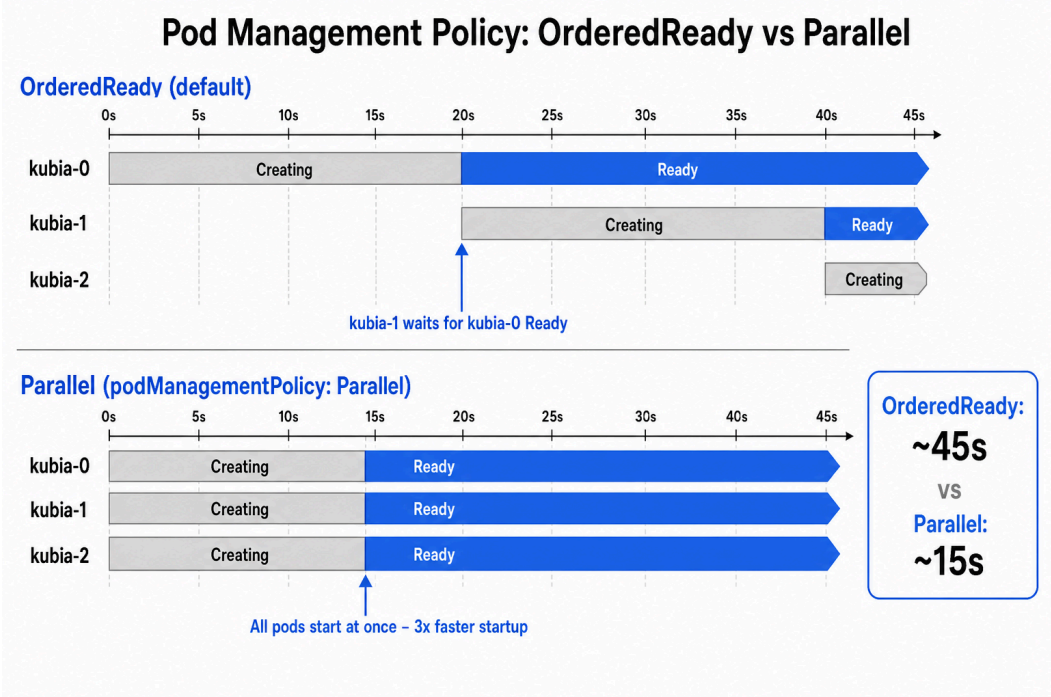
2판/현재 K8s의 대응

- `podManagementPolicy: Parallel` 추가 (K8s 1.7+)
- 생성·삭제만 병렬화, 업데이트는 여전히 역순 순차 (At-Most-One 보장 유지)

언제 써야 하나

- Cassandra·etcd처럼 순서 무관한 P2P 시스템
- MongoDB/MySQL primary-replica는 여전히 `OrderedReady` 권장

OrderedReady vs Parallel — 그림



모드	총 Ready 시간	권장 워크로드
OrderedReady (default)	~45s	master-slave, primary 우선 필요
Parallel	~15s	P2P (Cassandra 등)

Demo 7 — Parallel 모드 실측

관찰 포인트: t=9s에 3개 모두 Running. Demo 1(OrderedReady, ~45s)과 직접 비교

```
spec:
  podManagementPolicy: Parallel
```

```
--- t=9s ---
kubia-0    0/1    Running    0    3s
kubia-1    0/1    Running    0    3s
kubia-2    0/1    Running    0    3s          ← ★ 3개 모두 동시 Running 진입

--- t=15s ---
kubia-0    1/1    Running    0    9s
kubia-1    1/1    Running    0    9s
kubia-2    1/1    Running    0    9s          ← ~15s 만에 전체 Ready (OrderedReady ~45s 대비 3배)
```

무엇을 증명하나 — Parallel = 생성·삭제만 병렬, 업데이트는 그대로

- 3 replicas 부팅 시간: ~45s → ~15s (replicas 늘수록 차이 더 커짐)
- 단, `kubectl set image` rollout은 여전히 역순 순차(Demo 9 참고) → At-Most-One 보장 유지
- → Cassandra·etcd처럼 순서 무관 P2P엔 안전, MongoDB primary-replica엔 부적합

Demo 8 — 데이터 영속성 결정적 증거

이 시점까지 거친 시나리오:

1. Step 3: kuba-0에 1779075542 쓰기
2. Step 3-2: Pod 삭제 → 재생성
3. Step 5-6: Scale up 3 → Scale down 1
4. Step 7: StatefulSet 자체를 통째로 삭제
5. 완전히 다른 spec(Parallel + partition + startupProbe)으로 STS 재생성

```
$ curl http://kuba-0.kuba.default.svc:8080
You've hit kuba-0
Data stored on this pod: Data stored on kuba-0 at 1779075542
                               ↑★★★ 여전히 살아남음!
```

STS는 사라졌어도 PVC가 살아있었기에 새 Pod이 자동 재바인딩 → 데이터 복귀

StatefulSet 업데이트의 진화 — 1판 → 1.7 → 2판

1판 (K8s ~1.6): StatefulSet은 Pod Template 변경에 자동 업데이트 안 함

- `kubectl set image` 해도 기존 Pod은 그대로 — Pod Template만 갱신됨
- 사용자가 직접 Pod을 수동 삭제해야 새 spec으로 재생성됨
- → 분산 DB 업그레이드가 사람 손에 의존하는 위험한 운영

K8s 1.7 — `updateStrategy` 도입: 2가지 전략 선택 가능

- `OnDelete` (기본, 1판 동작 유지): Pod 수동 삭제해야 갱신
- `RollingUpdate` : 역순 자동 rollout (높은 ordinal부터)

K8s 1.7+ — `partition` 추가: RollingUpdate에 안전선

- `partition: N` → ordinal $\geq N$ 만 업데이트 (= Canary 보호선)
- → 다음 슬라이드에서 본격적으로 다룸

1판 → 2판 변화 ② — Partition Rolling Update

1판의 한계

- 1판은 StatefulSet 업데이트가 사실상 전체 rollout 한 방식
- 큰 클러스터(replicas=100)에서 새 버전 검증 불가
- "한 노드만 카나리 띄워보기"가 안 됨

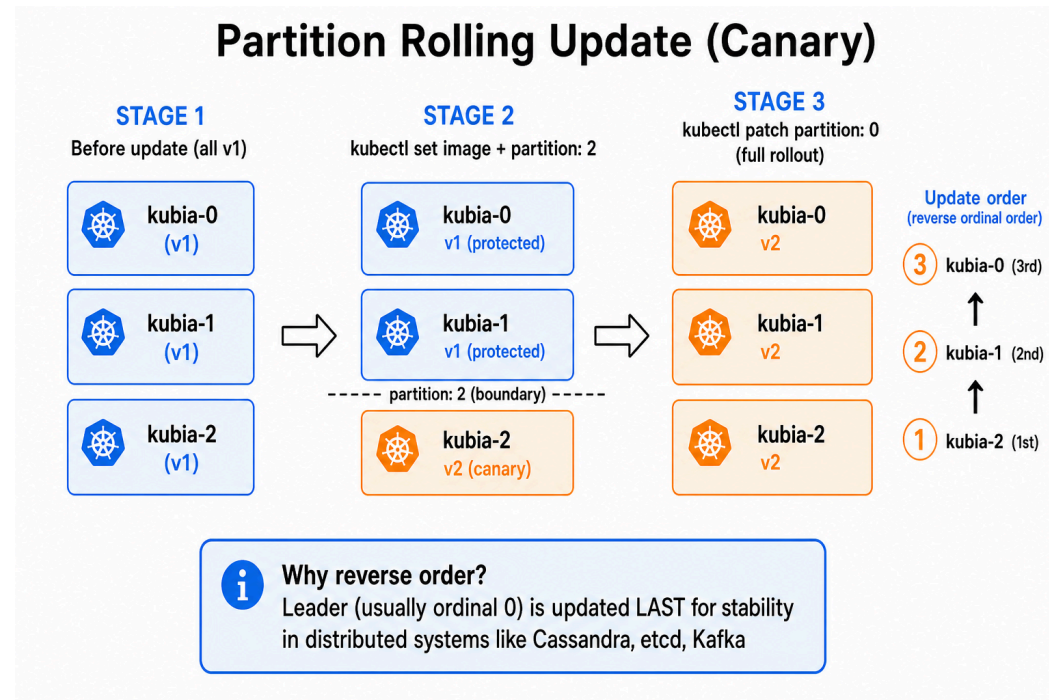
2판/현재 K8s의 대응

- `updateStrategy.rollingUpdate.partition` 필드 추가
- `partition: N` → ordinal $\geq N$ 만 업데이트 (보호선)
- 검증 후 `partition: 0` 으로 패치 → 전체 rollout

Deployment Canary와의 결정적 차이

- Deployment Canary: 어느 Pod이 카나리인지 클라이언트가 모름
- StatefulSet partition: 특정 ordinal 정확히 지정 (예: replicas=100 중 ordinal 99만)

Partition Rolling Update — 그림



업데이트 순서: ordinal 높은 쪽부터 → 낮은 쪽으로

Leader(보통 ordinal 0)를 마지막에 교체해 안정성 최대화

Demo 9-1: Canary 적용 (partition=2)

관찰 포인트: 이미지 변경했는데 ordinal 2 미만은 그대로 — partition0이 보호선

```
# v2 STS는 updateStrategy.rollingUpdate.partition: 2 로 설정
$ kubectl set image statefulset/kubia kubia=luksa/kubia-pet-peers
statefulset.apps/kubia image updated
```

```
--- t=15s ---
kubia-2 Pending luksa/kubia-pet-peers          ← ★ ordinal ≥2만 업데이트

=== final ===
kubia-0 → luksa/kubia-pet                      ← 보호선 (ordinal < 2)
kubia-1 → luksa/kubia-pet                      ← 보호선 (ordinal < 2)
kubia-2 → luksa/kubia-pet-peers                ← Canary (ordinal ≥ 2)
```

무엇을 증명하나 — Deployment Canary와의 결정적 차이

- Deployment: 어느 Pod이 카나리인지 클라이언트가 모름 (랜덤 hash 이름)
- StatefulSet: `kubia-2` 가 카나리임을 이름으로 정확히 지정 → 트래픽 라우팅·검증 쉬움
- replicas=100 운영 시: `partition: 99` → ordinal 99 한 대만 새 버전 → 프로덕션 영향 1%로 검증

Demo 9-2 (1/2): 설명 — 역순 업데이트 = Leader 보호

관찰 포인트: 다음 슬라이드 시간 흐름의 ordinal 순서

- t=15s → kuba-1 먼저 업데이트 (Canary였던 kuba-2 는 이미 v2)
- t=30s → kuba-0 마지막 업데이트
- 즉 kuba-2 → kuba-1 → kuba-0 (높은 ordinal → 낮은 ordinal)

무엇을 증명하나 — 역순 업데이트 전략의 안전 가치

- ordinal 0이 보통 Leader(MongoDB primary, etcd leader)이므로 **마지막에 교체**
- 새 버전에 버그가 있어도 kuba-1·2 실패 단계에서 rollout이 멈춤 → **Leader는 안전**
- → 분산 DB 운영의 표준 안전 패턴: "follower부터 점진 → leader는 검증 후 마지막"
- partition만 낮췄을 뿐 **이미지는 그대로** — 보호선 해제만으로 rollout 진행

Demo 9-2 (2/2): 데모 — 전체 Rollout (partition=0)

```
# Canary 검증 후
$ kubectl patch statefulset kubia -p \
  '{"spec":{"updateStrategy":{"rollingUpdate":{"partition":0}}}}'
```

```
--- t=15s ---
kubia-1 Pending luksa/kubia-pet-peers          ← ★ kubia-1 (2nd, 역순)

--- t=30s ---
kubia-0 Pending luksa/kubia-pet-peers          ← ★ kubia-0 (3rd, 마지막)

--- t=36s ---
kubia-0 Running luksa/kubia-pet-peers
kubia-1 Running luksa/kubia-pet-peers
kubia-2 Running luksa/kubia-pet-peers          ← 전체 v2 완료
```

kubia-2 → kubia-1 → kubia-0 역순 업데이트. Leader(0)가 마지막

1판 → 2판 변화 ③ — Pod Disruption Budget

1판의 한계

- 1판 출간 시점엔 노드 drain 시나리오가 일반적이지 않았음
- 클러스터 autoscaler·OS rolling 패치가 표준화되며 **자발적 중단**이 빈번해짐
- 안전장치 없으면 한 번의 **drain** 이 quorum 손실 → 분산 DB 다운

2판/현재 K8s의 대응

- **PodDisruptionBudget** 리소스 도입 (**policy/v1**)
- **minAvailable: N** 또는 **maxUnavailable: N** 으로 정책 정의
- API server가 eviction 요청 시 PDB 확인 → 정책 위반이면 거부

한계

- **자발적 중단**(drain, autoscaler)에만 적용
- 하드웨어 장애 같은 비자발적 중단은 막을 수 없음

PDB 정의 — 책의 권장 패턴

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: kubia-pdb
spec:
  minAvailable: 2           # 자발적 중단 시 최소 2개는 살아있어야
  selector:
    matchLabels:
      app: kubia
```

권장 운영 시나리오:

- 노드 OS rolling 패치 — 한 번에 한 노드씩 drain
- 클러스터 autoscaler — 비용 절감을 위해 노드 회수
- 노드 하드웨어 교체 — 계획된 작업

PDB 없으면 한 번의 명령으로 전체 quorum 손실 가능

Demo 10-1: PDB 적용 + 상태 확인

```
$ kubectl apply -f kubia-pdb.yaml
poddisruptionbudget.policy/kubia-pdb created

$ kubectl get pdb kubia-pdb
NAME           MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
kubia-pdb      2               N/A               1                     0s

$ kubectl describe pdb kubia-pdb
Name:           kubia-pdb
Namespace:      default
Min available:  2
Selector:       app=kubia
Status:
  Allowed disruptions: 1
  Current:             3
  Desired:             2
  Total:               3
```

ALLOWED DISRUPTIONS: 1 ← 현재 3개 Running, minAvailable=2 → 1개까지 evict 허용

Demo 10-2: Node Drain (안전한 중단)

```
$ kubectl drain minikube-m04 --ignore-daemonsets --delete-emptydir-data --force --timeout=30s
node/minikube-m04 cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/calico-node-d4sh2, kube-system/kube-proxy-58kg9
evicting pod default/kubia-1
pod/kubia-1 evicted                                     ← ★ PDB 허용 범위 내라 OK
node/minikube-m04 drained

$ kubectl get pods -l app=kubia -o wide
NAME      READY   STATUS             IP            NODE
kubia-0   1/1     Running            172.21.205.201 minikube-m02
kubia-1   0/1     ContainerCreating  <none>        minikube      ← ★ 재스케줄
kubia-2   1/1     Running            172.21.151.5   minikube-m03
```

새 위치에서도 같은 PVC **data-kubia-1** 재바인딩 → 데이터 손실 없음

1판 → 2판 변화 ④ — startupProbe & PreStop

1판의 한계

- 1판은 readiness/liveness probe 두 종류만 다룸
- 느린 부팅 앱(JVM, 분산 DB)이 liveness 실패로 무한 restart loop에 빠지는 패턴 다수 보고
- 종료 시점에 in-flight 요청이 503으로 떨어지는 문제 흔함

2판/현재 K8s의 대응

- **startupProbe** (K8s 1.16+): 부팅 동안 liveness/readiness 비활성화
- **PreStop** hook: SIGTERM 전 graceful shutdown 처리

사용해야 할 시그널

- 앱 부팅이 30초 이상 걸린다 → **startupProbe** 추가
- 종료 시 503 응답 보인다 → **preStop: sleep 5~15** 추가

startupProbe & PreStop 패턴

```
containers:
- name: kubia
  startupProbe:
    httpGet: { path: /, port: 8080 }
    failureThreshold: 30
    periodSeconds: 5
  readinessProbe: { ... }
  livenessProbe: { ... }
  lifecycle:
    preStop:
      exec:
        command: ["/bin/sh", "-c", "sleep 5"]
```

부팅 보호

최대 150초 부팅 허용

graceful shutdown

동작:

1. `startupProbe` 성공할 때까지 readiness/liveness 비활성
2. SIGTERM 전 `preStop` sleep으로 Endpoint 전파 대기
3. `terminationGracePeriodSeconds` (권장 30~60)와 함께 사용

1판 → 2판 변화 ⑤ — Operator 패턴

1판의 한계

StatefulSet으로 해결 안 되는 것:

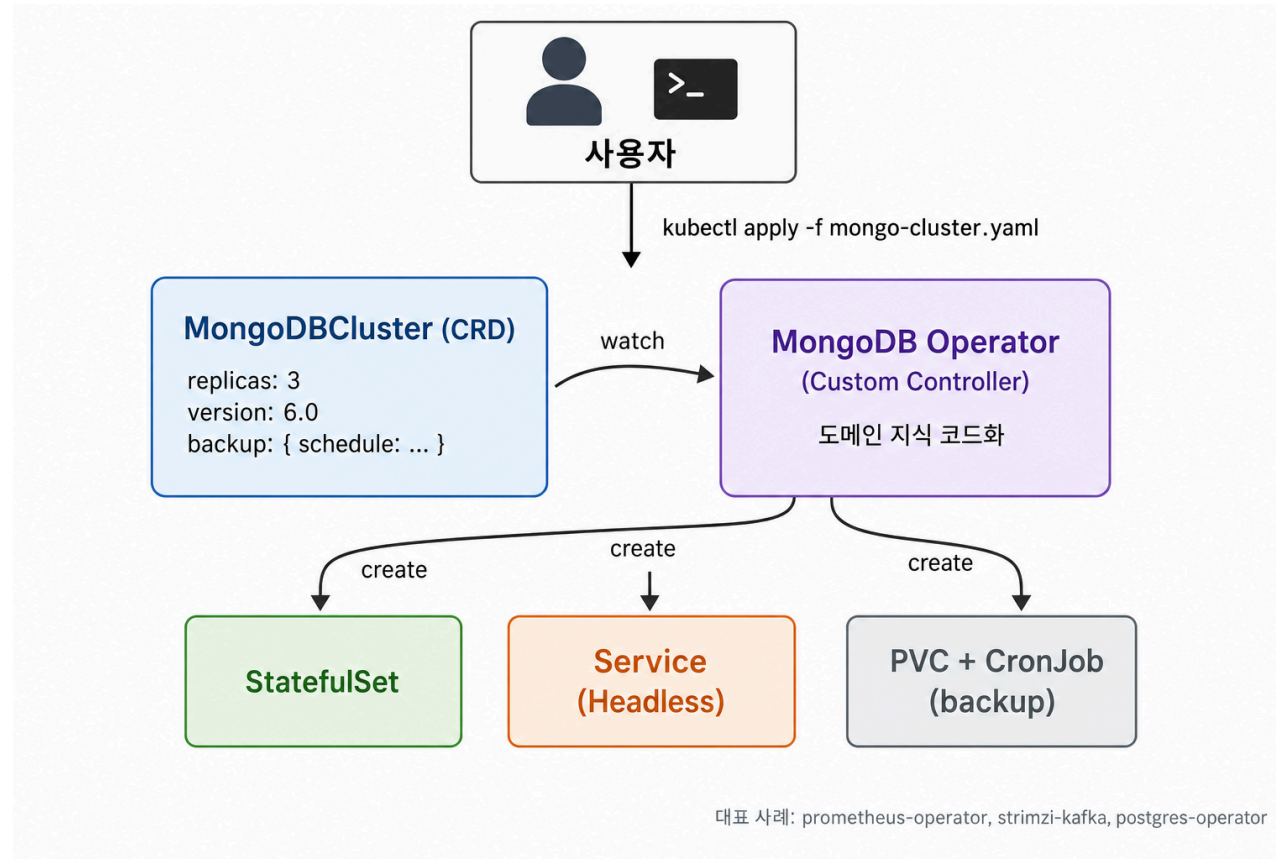
- 자동 backup/restore
- leader 죽으면 follower 자동 승격
- schema migration 자동 단계 실행
- 멤버 추가/제거 시 quorum 자동 재조정

→ 모두 도메인 지식(MongoDB election, Cassandra token, Kafka partition)이 필요

2판/현재 K8s의 대응

- Operator 패턴 표준화: `etcd-operator` (2016) → `prometheus-operator` → ...
- 구성: CRD(도메인 객체) + Custom Controller(K8s 기본 리소스 조작)
- Operator가 내부에서 StatefulSet을 만들고 관리함 → layering

Operator 구조 — 책 외 보강



- CRD: 사용자가 정의하는 도메인 객체 (예: MongoDBCluster)
- Operator: CRD를 watch하며 StatefulSet·Service·PVC를 조립

Demo 11 — Operator 흔적 확인

```
$ kubectl get crd
# 클러스터에 설치된 CRD 목록 - Operator가 등록한 CRD가 보임
```

대표적인 Operator CRD 예시:

<code>mongodbs.mongodb.com</code>	← MongoDB Operator
<code>kafkas.kafka.strimzi.io</code>	← Strimzi (Kafka)
<code>prometheuses.monitoring.coreos.com</code>	← prometheus-operator
<code>postgresqls.acid.zalan.do</code>	← Zalando postgres-operator

핵심 메시지:

- StatefulSet = low-level building block
- Operator = 그 위의 domain-specific orchestrator
- 둘은 대체가 아니라 layering

Deployment vs StatefulSet — 종합 비교

항목	Deployment	StatefulSet
Pod 이름	랜덤 hash	<code><sts>--<ordinal></code>
생성 순서	병렬	순차 (또는 Parallel)
업데이트 순서	임의 (maxSurge/maxUnavailable)	역순
Storage	공유 또는 없음	Pod별 전용 PVC
Scale down 시 PVC	N/A	보존
개별 Pod DNS	✗	<code><pod>.<svc>.<ns></code>
Canary	replicas 추가	partition (ordinal 지정)
적합 워크로드	무상태 웹/API	DB, 메시지 큐, 분산 캐시

실무 체크리스트

- ☒ `serviceName` 이 Headless Service 이름과 정확히 일치
- ☒ Service에 `clusterIP: None` (가장 흔한 실수)
- ☒ `volumeClaimTemplate` storage 크기 충분 (resize 어려움)
- ☒ StorageClass `WaitForFirstConsumer` (멀티존)
- ☒ PDB 정의로 quorum 보호
- ☒ `terminationGracePeriodSeconds` + `preStop` 설정
- ☒ DNS 캐싱 주의 (JVM `networkaddress.cache.ttl=30`)

흔한 함정 5가지

1. `clusterIP: None` 누락 → 개별 Pod DNS 작동 안 함
2. PVC 수동 삭제 시 PV도 함께 사라짐 (RECLAIM POLICY=Delete 기본)
3. 같은 이름으로 STS 재생성하면 기존 PVC 재바인딩 (의도면 OK, 깨끗한 시작 원하면 PVC도 삭제)
4. `--force --grace-period=0` 남용 → split-brain 가능성
5. Parallel + readiness probe 부재 → bootstrap 전 트래픽 유입

정리 — 10장 핵심 한 페이지

- StatefulSet = Stable Identity + Stable Storage + Ordered Ops
- Headless Service + `clusterIP: None` = peer discovery
- volumeClaimTemplate = Pod별 PVC 자동, 데이터 영속
- Parallel (2판) = 빠른 시작, 단 순서 무관 시스템만
- partition (2판) = 정확한 ordinal canary
- PDB (2판) = 자발적 중단에서 quorum 보호
- startupProbe/PreStop (2판) = 부팅·종료 안전성
- Operator = StatefulSet 위에 도메인 자동화

Q & A

다음 장 — Chapter 11. Understanding Kubernetes Internals